IssueVault: Hostel Reporting and Monitoring System

A Minor Project Report

Submitted in partial fulfillment of requirement of the

Degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

Meena

EN23CS301609

Under the Guidance of

Dr. Sheetal Bawane

Dr. Garima S Tukra

Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE- 453331

APRIL-2026

IssueVault: Hostel Reporting and Monitoring System

A Minor Project Report

Submitted in partial fulfillment of requirement of the

Degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

Meena
EN23CS301609

Under the Guidance of
Dr. Sheetal Bawane
Dr. Garima S Tukra

Department of Computer Science & Engineering
Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE- 453331

APRIL-2026

Report Approval

The project work **“IssueVault: Hostel Reporting and Monitoring System”** is hereby approved as a creditable study of an engineering subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn therein; but approve the “Project Report” only for the purpose for which it has been submitted.

Internal Examiner 1

Dr. Sheetal Bawane

Assistant Professor

Medicaps University

Internal Examiner 2

Dr. Garima S Tukra

Assistant Professor

Medicaps University

Declaration

I hereby declare that the project entitled **"IssueVault: Hostel Reporting and Monitoring System"** submitted in partial fulfillment for the award of the degree of Bachelor of Technology in 'Computer Science and Engineering' completed under the supervision of

Dr. Sheetal Bawane, Assistant Professor, CSE &

Dr. Garima S Tukra, Assistant Professor, CSE

Faculty of Engineering, Medi-Caps University Indore is an authentic work.

Further, I declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Signature and name of the student with date

Certificate

We, **Dr. Sheetal Bawane** and **Dr. Garima S Tukra** certify that the project entitled **"IssueVault: Hostel Reporting and Monitoring System"** submitted in partial fulfillment for the award of the degree of Bachelor of Technology by **Meena** is the record carried out by her under my/our guidance and that the work has not formed the basis of award of any other degree elsewhere.

Dr. Sheetal Bawane

Computer Science & Engineering

Medi-Caps University, Indore

Dr Garima S Tukra

Computer Science & Engineering

Medi-Caps University, Indore

Dr. Kailash Chandra Bandhu

Head of the Department

Computer Science & Engineering

Medi-Caps University, Indore

Acknowledgements

I would like to express my deepest gratitude to the Honorable Chancellor, **Shri R C Mittal,** who has provided me with every facility to successfully carry out this project, and my profound indebtedness to **Prof. (Dr.) D. K. Patnaik,** Vice Chancellor, Medi-Caps University, whose unfailing support and enthusiasm has always boosted up my morale. I also thank **Dr. Ratnesh Litoriya,** Dean, Faculty of Engineering, Medi-Caps University, for giving me a chance to work on this project. I would also like to thank my Head of the Department **Dr. Kailash Chandra Bandhu** for his continuous encouragement for the betterment of the project.

It is their help and support, due to which we became able to complete the design and technical report.

Without their support this report would not have been possible.

Meena
B.Tech. III Year (K)
Department of Computer Science & Engineering
Faculty of Engineering
Medi-Caps University, Indore

Abstract

Issue reporting in hostel environments is often handled through manual or informal methods, which leads to delays, lack of transparency, and inefficient tracking of complaints. Students face difficulties in reporting issues effectively, while administrators struggle to monitor and resolve them in a timely manner, resulting in poor communication and delayed resolution.

To address these challenges, IssueVault – Hostel Issue Reporting and Monitoring System is proposed as a web-based solution that digitizes and streamlines the complaint reporting and tracking process. The system allows students to submit complaints through a structured interface and enables administrators to monitor and update the status of reported issues efficiently. Each complaint is recorded with a timestamp and can be tracked throughout its lifecycle, ensuring accountability and transparency.

The system is developed using HTML, CSS, and JavaScript for the frontend, Node.js and Express.js for backend processing, and MongoDB for data storage. It follows a modular and scalable architecture, allowing future enhancements such as real-time notifications and enhanced authentication mechanisms.

The implementation of IssueVault improves communication between users and administrators, reduces manual effort, and ensures faster resolution of issues, making the system efficient and reliable for institutional use.

Keywords:

Issue Reporting, Complaint Tracking, Web Application, Hostel System, Node.js, MongoDB, Transparency, Complaint Status, User Authentication

Table of Contents

	Content	Page No.
	Report Approval	
	Declaration	
	Certificate	
	Acknowledgement	
	Abstract	
	Table of Contents	
	List of figures	
	List of tables	
	Abbreviations	
	Notations & Symbols	
Chapter 1	Introduction (Whichever is applicable)	1 - 3
	1.1 Introduction	
	1.2 Literature Review	
	1.3 Objectives	
	1.4 Significance	
	1.5 Research Design	
	1.6 Source of Data	
	1.7 Chapter Scheme	
Chapter 2	REQUIREMENTS SPECIFICATION	4 - 6
	2.1 User Characteristics	
	2.2 Functional Requirements	
	2.3 Dependencies	
	2.4 Performance Requirements	
	2.5 Hardware Requirements	
	2.6 Constraints & Assumptions	
Chapter 3	DESIGN (Whichever is applicable)	7 - 15
	3.1 Algorithm (if Applicable)	
	3.2 Function Oriented Design for procedural approach	
	3.3 **System Design (**Whichever is applicable)	
	3.3.1 Data Flow Diagrams (Level 0,Level1)	
	3.3.2 Activity Diagram	

	3.3.3 Flow Chart	
	3.3.4 Class Diagram	
	3.3.5 ER Diagram	
	3.3.6 Sequence diagram	
	3.4 **Database Design**	
	3.4.1 Logical Database Design	
	3.4.2 Physical Database Design	
Chapter 4	Implementation, Testing, and Maintenance	16 - 21
	4.1 Introduction to Languages, IDE's, Tools and Technologies used for Implementation	
	4.2 Testing Techniques and Test Plans (According to project)	
	4.3 Installation Instructions	
	4.4 End User Instructions	
Chapter 5	Results and Discussions	17 - 21
	5.1 User Interface Representation (of Respective Project)	
	5.2 Brief Description of Various Modules of the system	
	5.3 Snapshots of system with brief detail of each	
	5.4 Back Ends Representation (Database to be used)	
	5.5 Snapshots of Database Tables with brief description	
Chapter 6	Summary and Conclusions	22
Chapter 7	Future scope	23 - 24
	Appendix	
	Bibliography	

List of Figures

Figure No.	Title
Figure 3.3.1(0)	Data Flow Diagram (Level 0)
Figure 3.3.1(1)	Data Flow Diagram (Level 1)
Figure 3.3.2	Activity Diagram
Figure 3.3.3	Flowchart (System Flow)
Figure 3.3.4	Class Diagram
Figure 3.3.5	ER Diagram
Figure 3.3.6	Sequence Diagram
Figure 5.1	Login Page Interface
Figure 5.2	Registration Page Interface
Figure 5.3	Complaint Submission Page
Figure 5.4	Admin Dashboard
Figure 5.5	Complaint Status Page

List of Tables

Table No.	Title
Table 2.1	User Characteristics
Table 2.2	Functional Requirements
Table 2.5	Hardware Requirements
Table 3.2	Function Oriented Design
Table 4.2	Testing Plan
Table 3.4.2	Database Tables Description

Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
UI	User Interface
DB	Database
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
JWT	JSON Web Token
CRUD	Create, Read, Update, Delete
URL	Uniform Resource Locator
IDE	Integrated Development Environment
OS	Operating System
JSON	JavaScript Object Notation
MVC	Model View Controller

Notations & Symbols

Symbol / Notation	Meaning
→	Flow of data or process
1:M	One-to-Many relationship
PK	Primary Key
FK	Foreign Key
<<include>>	Mandatory relationship in Use Case Diagram
<<extend>>	Optional relationship in Use Case Diagram
()	Represents a process or activity
{}	Represents a group or block
[]	Represents a system/component

Chapter - 1

INTRODUCTION

1.1 Introduction

In hostel and institutional environments, students often face various issues related to facilities such as maintenance, cleanliness, and basic services. Traditionally, these issues are reported manually or through informal communication methods, which leads to delays, lack of proper tracking, and poor communication between students and administrators.

To overcome these challenges, a digital system is required that allows users to report issues in a structured and efficient manner. The system should also provide a way to track the status of complaints and ensure timely updates.

IssueVault – Hostel Issue Reporting and Monitoring System is developed as a web-based application that simplifies the process of complaint reporting and tracking. It enables students to submit issues through an easy-to-use interface and allows administrators to view, monitor, and update the status of complaints.

The system maintains a centralized database where all complaints are stored along with timestamps, ensuring transparency and accountability. It improves communication, reduces manual effort, and helps in faster resolution of issues.

1.2 Literature Review

Issue reporting systems have been studied and implemented in various domains such as educational institutions, corporate environments, and public service platforms. Earlier approaches relied on manual methods like complaint registers and verbal reporting, which often resulted in delays, lack of proper tracking, and limited transparency.

With the advancement of web technologies, digital complaint systems have been introduced that allow users to submit issues online and track their status. These systems improve communication and provide a structured approach to handling complaints. However, many existing solutions lack real-time updates, efficient validation mechanisms, and proper tracking throughout the issue lifecycle. Modern systems utilize technologies such as Node.js, Express.js, and MongoDB to provide better performance, scalability, and efficient data handling. These technologies enable the development of user-friendly and responsive applications with secure authentication and organized data storage.

The proposed system, IssueVault, is designed by considering the limitations of existing systems and aims to provide a simple, efficient, and transparent platform for issue reporting and tracking in a hostel environment.

1.3 Objectives

The main objectives of the IssueVault system are:

- To provide a platform for students to submit complaints easily
- To enable administrators to view and update complaint status
- To ensure transparency in issue tracking
- To maintain a structured record of all complaints
- To reduce manual effort in complaint handling
- To improve communication between students and administrators

1.4 Significance

The IssueVault system is significant as it provides an efficient and structured approach to issue reporting in hostel environments. It replaces traditional manual methods with a digital platform, making the process faster and more organized.

The system improves transparency by allowing users to track the status of their complaints at every stage. It also enhances communication between students and administrators, ensuring that issues are addressed in a timely manner.

Additionally, the system maintains a centralized record of all complaints, which helps in future reference and analysis. Overall, IssueVault contributes to improved efficiency, reliability, and user satisfaction.

1.5 Research Design

The development of the IssueVault system follows a structured approach consisting of the following steps:

- Requirement analysis to identify system needs and objectives
- System design including architecture, database schema, and UML diagrams
- Implementation using HTML, CSS, JavaScript, Node.js, and MongoDB
- Testing of individual modules and overall system functionality
- Documentation of system design and implementation
- Evaluation to ensure the system meets all requirements

1.6 Source of Data

The data used in the IssueVault system is primarily collected from user inputs and system interactions. The following are the main sources of data:

- User-provided data such as registration details and login credentials
- Complaint data submitted by students (title, description, etc.)

- Status updates provided by administrators
- System-generated data such as timestamps for complaint submission and updates

1.7 Chapter Scheme

The report is organized into multiple chapters, each focusing on a specific aspect of the system development process.

Chapter 1: Introduction

This chapter provides an overview of the project, including background, problem statement, objectives, significance, research design, and source of data.

Chapter 2: Requirements Specification

This chapter describes the functional and non-functional requirements of the system. It also includes user characteristics, system constraints, dependencies, and performance requirements.

Chapter 3: Design

This chapter focuses on the system design, including architectural design, database design, and various UML diagrams such as data flow diagrams, activity diagrams, class diagrams, and sequence diagrams.

Chapter 4: Implementation, Testing, and Maintenance

This chapter explains the technologies used for development, the implementation process, testing techniques, and system maintenance aspects.

Chapter 5: Results and Discussions

This chapter presents the user interface, system modules, and outputs with screenshots and explanations. It also includes backend representation and database details.

Final Sections

The report concludes with summary, conclusions, future scope, appendices, and references.

Chapter - 2

REQUIREMENT SPECIFICATION

2.1 User Characteristics

The IssueVault system is designed for two types of users with different roles and characteristics.

User Type	Characteristics	System Interaction
Student	Basic computer knowledge, familiar with web interfaces	Can register, log in, submit complaints, and view complaint status
Administrator	Moderate technical knowledge, higher access privileges	Can log in, view all complaints, and update complaint status

The system is designed to be simple and user-friendly so that both students and administrators can use it efficiently without requiring advanced technical knowledge.

2.2 Functional Requirements

Req ID	Requirement Description
FR-1	The system shall allow users to register with required details
FR-2	The system shall validate user input during registration
FR-3	The system shall allow users to log in using valid credentials
FR-4	The system shall verify user credentials before granting access
FR-5	The system shall display error messages for invalid login attempts
FR-6	The system shall allow students to submit complaints
FR-7	The system shall validate complaint details before submission
FR-8	The system shall store complaint data in the database
FR-9	The system shall display confirmation after successful submission
FR-10	The system shall allow administrators to view all complaints

FR-11	The system shall display complaint details with status and timestamp
FR-12	The system shall allow administrators to update complaint status
FR-13	The system shall store updated status in the database
FR-14	The system shall reflect updated status to users
FR-15	The system shall allow users to track complaint status
FR-16	The system shall allow users to verify the updated status of their complaints

2.3 Dependencies

The IssueVault system depends on various technologies and external factors for proper functioning. These dependencies ensure smooth communication between system components and reliable performance.

- The system depends on the Node.js runtime environment for backend execution
- It requires MongoDB for storing and retrieving user and complaint data
- A web browser is required to access the system interface
- A stable internet connection is necessary for client-server communication
- Proper availability of system resources such as memory and processing power is required

2.4 Performance Requirements

The IssueVault system is designed to provide efficient and reliable performance under normal operating conditions. The system should respond quickly to user actions and handle multiple requests without significant delays.

- The system should respond to user requests within 2–3 seconds
- The system should support multiple users accessing it simultaneously
- Database operations should be optimized for fast data retrieval and storage
- The system should maintain consistent performance under normal load conditions
- Delays during complaint submission and status updates should be minimized

2.5 Hardware Requirements

Component	Requirement
Processor	Basic processor (Intel i3 or above recommended)
RAM	Minimum 4 GB
Storage	At least 500 MB free space
Input Devices	Keyboard and Mouse
Output Devices	Monitor/Display
Network	Internet connection required

The system does not require any specialized hardware and can run on standard computing devices with basic configuration. A stable internet connection is necessary for accessing the web-based application.

2.6 Constraints & Assumptions

Constraints

The IssueVault system is developed under certain limitations that may affect its design and implementation:

- The system is limited to a web-based environment
- It depends on internet connectivity for proper functioning
- Security is implemented at a basic level (advanced features not included)
- The system is designed for a limited user base (hostel/institution level)
- Development is carried out within limited time and resources

Assumptions

The system is designed based on the following assumptions:

- Users have basic knowledge of using web applications
- Users have access to a stable internet connection
- The system will be used within a controlled environment (hostel/institution)
- Required software and hardware resources are available
- Users will provide valid and accurate input data

Chapter - 3

DESIGN

3.1 Algorithm

Since IssueVault is not a mathematical or computation-heavy system, a full algorithm is not strictly required. However, a basic logical flow of operations can be described for key functionalities.

Algorithm: Complaint Submission and Processing

1. Start
2. User registers or logs into the system
3. System validates user credentials
4. If authentication is successful, user proceeds
5. User enters complaint details
6. System validates input data
7. If data is valid, complaint is stored in database
8. System generates timestamp for the complaint
9. Confirmation message is displayed to the user
10. Administrator views complaint
11. Administrator updates complaint status
12. System saves updated status
13. Updated status is reflected to the user
14. End

3.2 Function Oriented Design

Module	Description
User Authentication Module	Handles user registration and login verification
Complaint Module	Allows users to submit complaints with required details
Admin Module	Enables viewing and updating of complaints
Complaint Tracking Module	Allows users to check status of submitted complaints
Database Module	Stores and retrieves user and complaint data

3.3 System Design

The system design of IssueVault describes the overall structure of the system, data flow, and interaction between different components. It helps in understanding how the system processes user requests and manages data.

3.3.1 Data Flow Diagrams (DFD)

Level 0 DFD (Context Diagram)

The system is represented as a single process interacting with external entities.

Fig 3.3.1(0)

External Entities:

- Student (User)
- Administrator

Data Flow:

- Student → submits complaint → System
- System → sends status → Student
- Administrator → updates complaint → System
- System → provides complaint data → Administrator

Level 1 DFD

This level breaks the system into major processes:

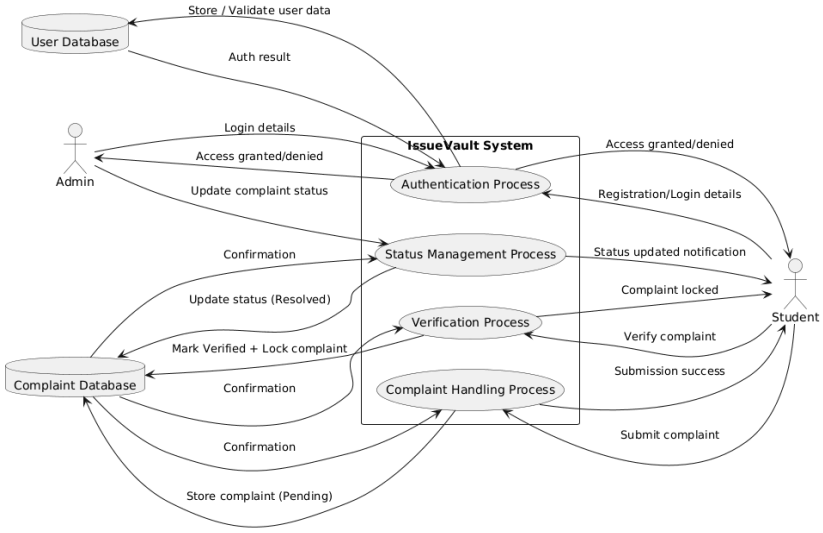


Fig 3.3.1(1)

- The Level 1 DFD of the IssueVault system shows a detailed breakdown of the system into major processes and data flows.
- The system consists of four main processes: **Authentication, Complaint Handling, Status Management, and Verification**.
- In the **Authentication Process**, students and administrators enter their login or registration details, which are validated using the User Database before granting access.
- In the **Complaint Handling Process**, students submit complaint details, which are checked and stored in the Complaint Database with status set to "Pending".
- In the **Status Management Process**, the administrator updates the complaint status (for example, marking it as "Resolved"), and the updated information is stored in the database.
- In the **Verification Process**, the student verifies whether the complaint has been resolved. After verification, the system updates the status to "Verified" and locks the complaint.
- The system interacts with two main data stores: **User Database** and **Complaint Database**.
- This diagram clearly represents how data flows between users, processes, and databases, ensuring proper tracking and handling of complaints in the system.

3.3.2 Activity Diagram

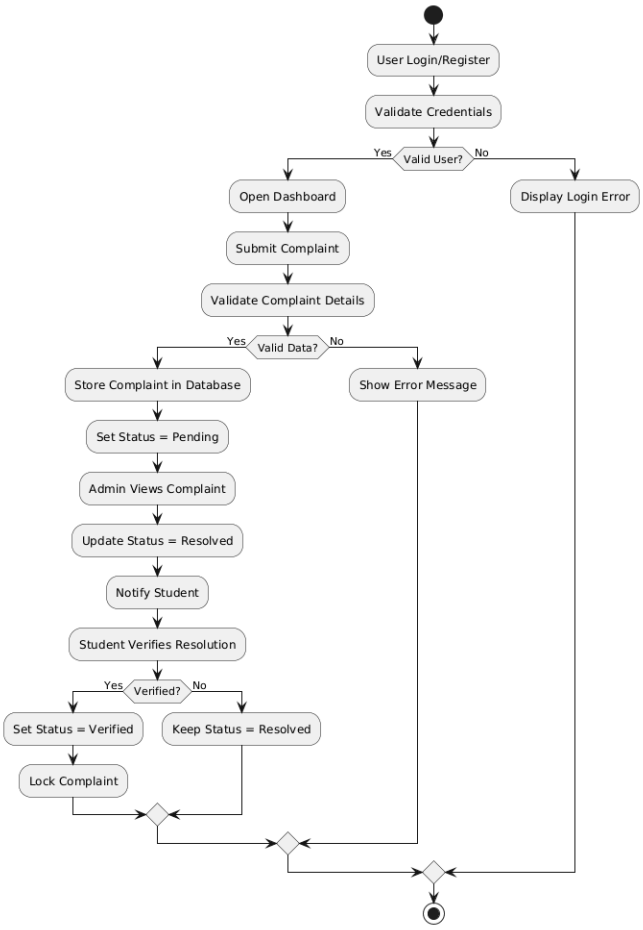


Fig 3.3.2

The activity diagram represents the workflow of the IssueVault system from user login to final complaint closure. After successful authentication, the user submits a complaint which is validated and stored in the database with a “Pending” status. The administrator reviews the complaint and marks it as “Resolved” once the issue is addressed.

The system then notifies the student for verification. If the student confirms that the issue is resolved, the complaint status is updated to “Verified” and the record is locked to prevent any further modifications. If not verified, the complaint remains in the “Resolved” state. This process ensures transparency and accountability in the system.

3.3.3 Flowchart

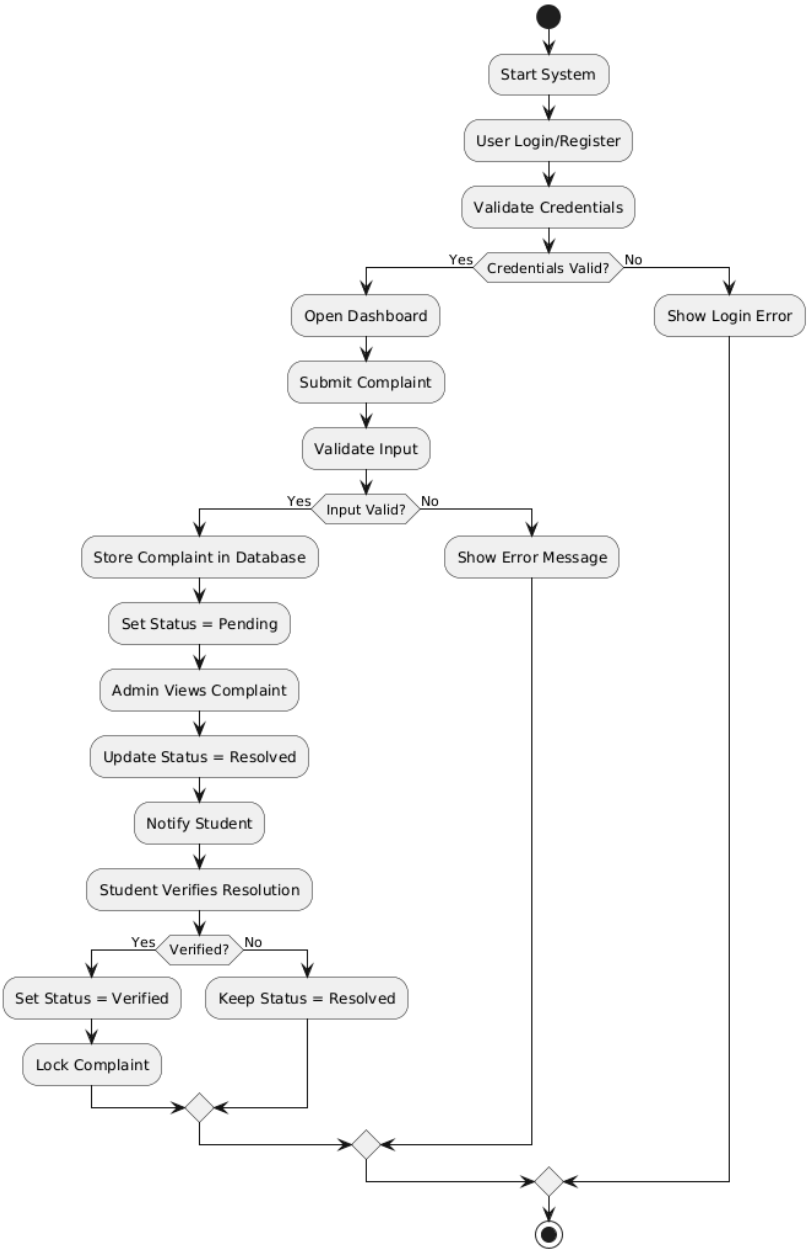


Fig 3.3.3

- The flowchart represents the working flow of the IssueVault system.
- The process starts with user login or registration and credential validation.
- If valid, the user accesses the dashboard and submits a complaint.
- The system validates the input and stores the complaint with status “Pending”.
- The administrator reviews the complaint and marks it as “Resolved”.
- The system notifies the student after resolution.
- The student verifies whether the issue is resolved.
- If verified, the complaint status is updated to “Verified” and locked.
- If not verified, the status remains “Resolved”.
- The process ends after the final status update.

3.3.4 Class Diagram

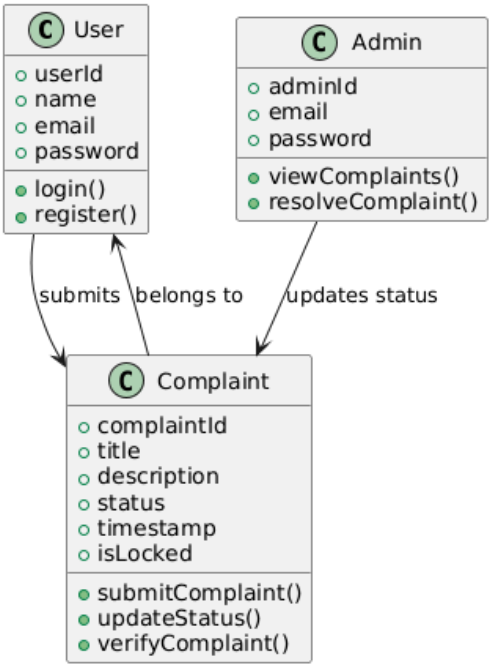


Fig 3.3.4

- The class diagram represents the structure of the IssueVault system using object-oriented design.
- The User class contains user details and handles login and registration operations.
- The Complaint class stores complaint details such as title, description, status, timestamp, and lock state. It also handles submission, status update, and verification.
- The Admin class is responsible for viewing complaints and updating their status after resolution.
- A User can submit multiple complaints (one-to-many relationship).

- Each complaint belongs to a specific user.
- The Admin interacts with complaints to update their status based on resolution.

3.3.5 ER Diagram

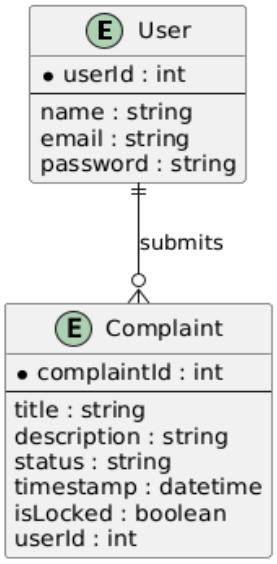


Fig 3.3.5

- The ER diagram represents the database structure of the IssueVault system.
- There are two main entities: User and Complaint.
- The User entity stores user details such as userId, name, email, and password.
- The Complaint entity stores complaint details such as title, description, status, timestamp, and lock status.
- Each complaint is linked to a user using userId as a foreign key.
- A single user can submit multiple complaints, showing a one-to-many relationship.
- This structure ensures proper organization and easy retrieval of data from the database.

3.3.6 Sequence Diagram

- The sequence diagram shows the interaction between User, Admin, System, and Database in the IssueVault system.
- The process starts when the user logs in and the system validates credentials with the database.
- After successful login, the user submits a complaint which is validated and stored with status “Pending”.
- The administrator retrieves and views all complaints from the database.

- The administrator then updates the complaint status to “Resolved” after addressing the issue.
- The system notifies the user about the resolution.
- The user verifies the complaint, confirming whether the issue is resolved.
- If verified, the system updates the status to “Verified” and locks the complaint, preventing further changes.
- This ensures a complete and transparent workflow from submission to final closure.

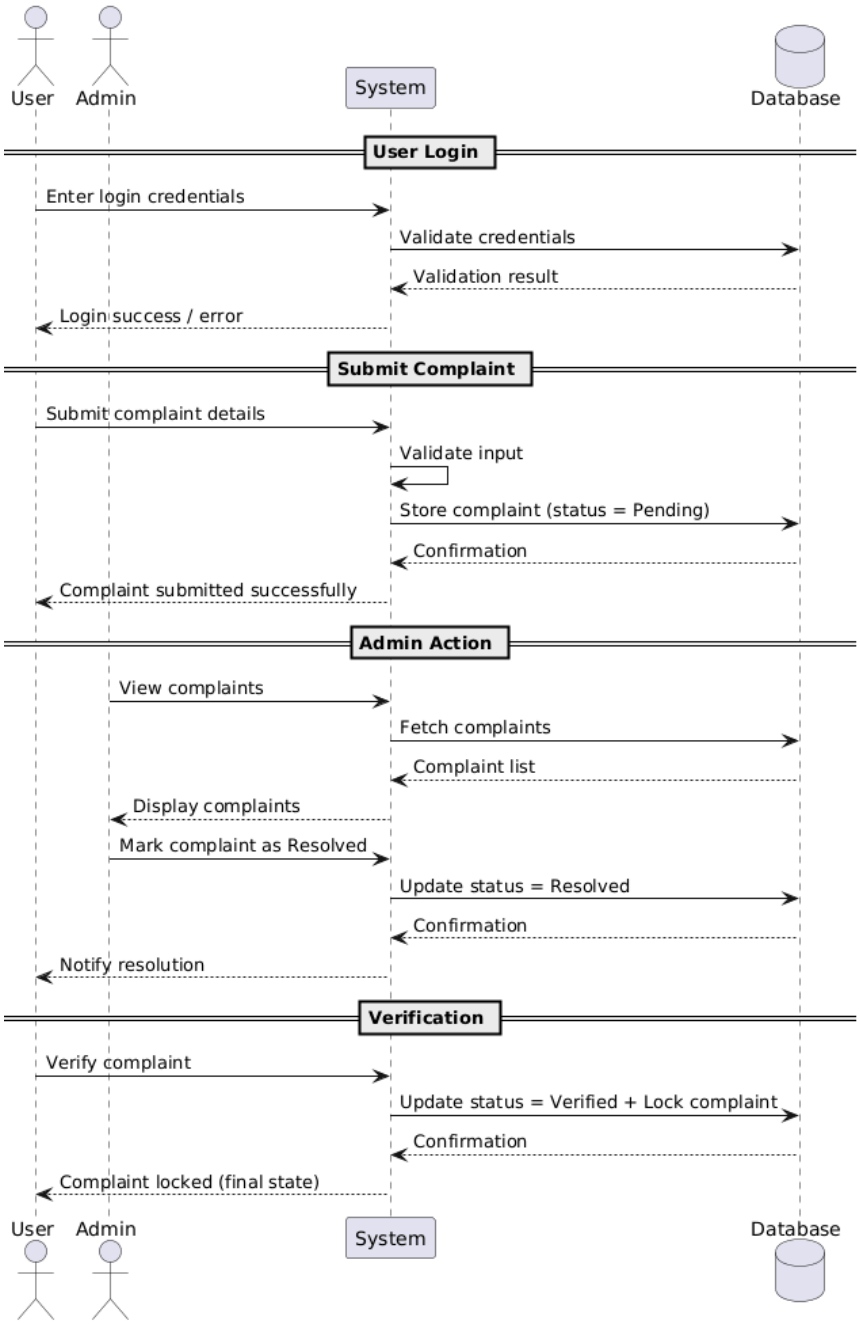


Fig 3.3.6

3.4 Database Design

The database design of IssueVault defines how data is logically structured and physically stored in MongoDB. It ensures efficient storage, retrieval, and management of user and complaint information.

3.4.1 Logical Database Design

The logical design describes the structure of data in terms of entities, attributes, and relationships.

Entities:

1. User: Stores information about students and administrators.

Attributes:

- userId (Primary Key)
- name
- email
- password
- role (student/admin)

2. Complaint: Stores details of issues submitted by users.

Attributes:

- complaintId (Primary Key)
- title
- description
- status (Pending / Resolved / Verified)
- timestamp
- isLocked (Boolean)
- userId (Foreign Key)

Relationship:

- One User can submit multiple Complaints (1:M relationship)

3.4.2 Physical Database Design

The physical design represents how data is actually stored in MongoDB collections.

User Collection

Field Name	Type	Description
user_id	ObjectId	Unique identifier for user
name	String	Name of user
email	String	Unique email address
password	String	Encrypted password
role	String	User type (Student/Admin)

Complaint Collection

Field Name	Type	Description
complaint_id	ObjectId	Unique complaint ID
title	String	Complaint title
description	String	Detailed issue
status	String	Complaint status
timestamp	DateTime	Submission time
user_id	ObjectId	Links to user
isLocked	Boolean	True after verification
userId	ObjectId	Reference to User

Chapter - 4

4. IMPLEMENTATION, TESTING, AND MAINTENANCE

4.1 Introduction to Languages, IDEs, Tools and Technologies Used for Implementation

The IssueVault system is developed using modern web development technologies to ensure efficiency, scalability, and ease of use.

Frontend Technologies

- HTML: Used to structure the web pages.
- CSS: Used for styling and improving UI design.
- JavaScript: Used for client-side validation and interactivity.

Backend Technologies

- Node.js: Used as the runtime environment for server-side execution.
- Express.js: Used for handling routing and API requests.

Database

- MongoDB: Used for storing user and complaint data in a flexible NoSQL format.

Tools Used

- Visual Studio Code: Code editor used for development.
- Postman: Used for testing APIs.
- MongoDB Compass: Used for database visualization and management.
- Google Chrome: Used for testing the application interface.

4.2 Testing Techniques and Test Plans

The system is tested to ensure that all functionalities work correctly and efficiently.

Testing Techniques Used

- **Unit Testing:** Each module (login, complaint submission, status update) is tested individually.
- **Integration Testing:** Ensures proper interaction between frontend, backend, and database.
- **System Testing:** The complete system is tested as a whole to verify overall functionality.
- **Validation Testing:** Ensures that all inputs are correctly validated before processing.

Test Plan Table

Test Case	Description	Expected Result
TC-1	User login with valid credentials	Access granted
TC-2	User login with invalid credentials	Error message displayed
TC-3	Submit complaint with valid data	Complaint stored successfully
TC-4	Submit empty complaint form	Validation error shown
TC-5	Admin updates complaint status	Status updated in database
TC-6	User verifies complaint	Status set to Verified and locked

4.3 Installation Instructions

To run the IssueVault system, the following steps are required:

- Install Node.js on the system.
- Install MongoDB and start the database server.
- Open the project folder in Visual Studio Code.
- Run backend server using: node server.js
- Open the frontend in a browser or use localhost URL.
- Ensure MongoDB is running before starting the application.

4.4 End User Instructions

The system is designed to be simple and user-friendly.

For Students:

- Register or log in to the system.
- Submit complaints using the complaint form.
- View status of submitted complaints.
- Verify the complaint once it is resolved.

For Administrators:

- Log in with admin credentials.
- View all submitted complaints.
- Update complaint status after resolution.
- Monitor verified and pending complaints.

Chapter - 5

RESULTS AND DISCUSSIONS

5.1 User Interface Representation

The IssueVault system provides a simple and user-friendly interface designed for both students and administrators.

- **Login Page:**
  The login page allows users to securely access the system using their credentials. It ensures authentication before granting access to system features.
- **Complaint Submission Page:**
  This page allows students to submit complaints by entering title and description. Proper validation is applied to ensure correct input.
- **Admin Dashboard:**
  The admin dashboard displays all submitted complaints with their current status. It allows administrators to review and update complaints.
- **Complaint Status Page:**
  This page allows students to track the status of their complaints and verify resolution.

5.2 Brief Description of Various Modules of the System

The IssueVault system is divided into several modules, each responsible for a specific function. These modules work together to ensure smooth operation of the system.

1. Authentication Module

This module handles user registration and login. It verifies user credentials before allowing access to the system. It ensures that only authorized users can use the application.

2. Complaint Module

This module allows students to submit complaints through a structured form. It validates input data and stores complaint details in the database along with a timestamp for tracking purposes.

3. Admin Module

This module enables administrators to view all submitted complaints. It allows them to update complaint status after resolving issues and ensures proper monitoring of all reported problems.

4. Verification Module

This module allows students to verify whether their complaint has been resolved. Once verified, the complaint status is updated and locked to prevent further changes.

5. Database Module

This module is responsible for storing and retrieving all system data. It maintains user information and complaint records using MongoDB, ensuring data consistency and easy access.

5.3 Snapshots of System with Brief Detail

This section presents the user interface of the IssueVault system. The snapshots show how different users interact with the system and how various functionalities are implemented.

Figure 5.1: Login Page Interface

The login page allows registered users (students and administrators) to securely access the system by entering valid credentials. It performs authentication before granting access to the dashboard.

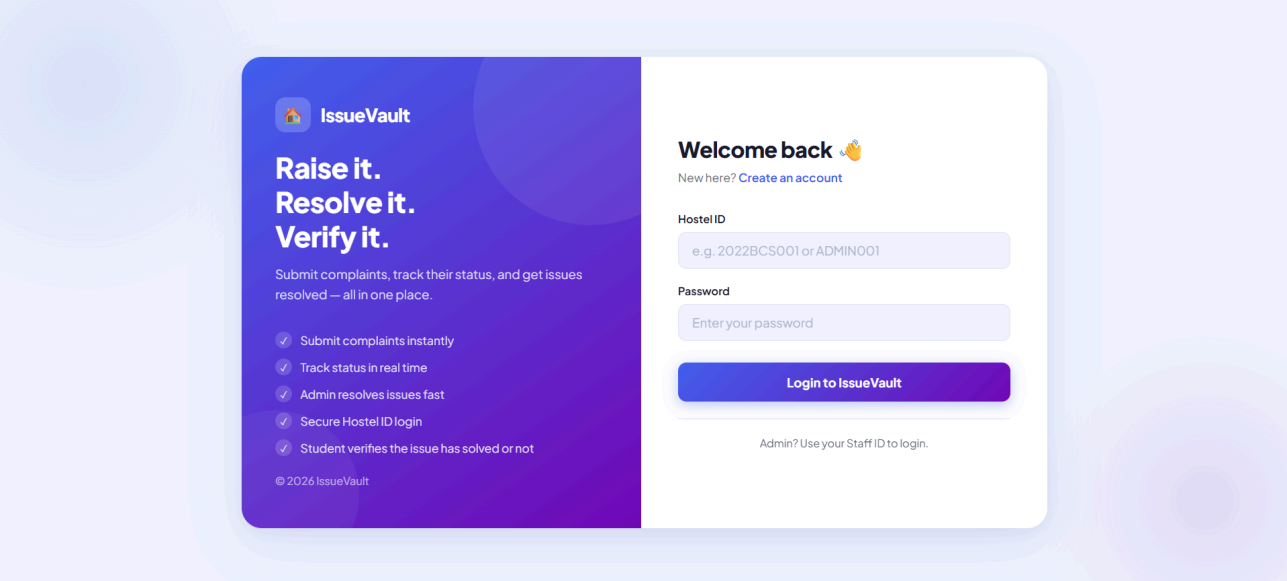


Figure 5.2: Registration Page Interface

The registration page allows new users to create an account by providing necessary details such as name, email, and password. The system validates input before creating a new user account.

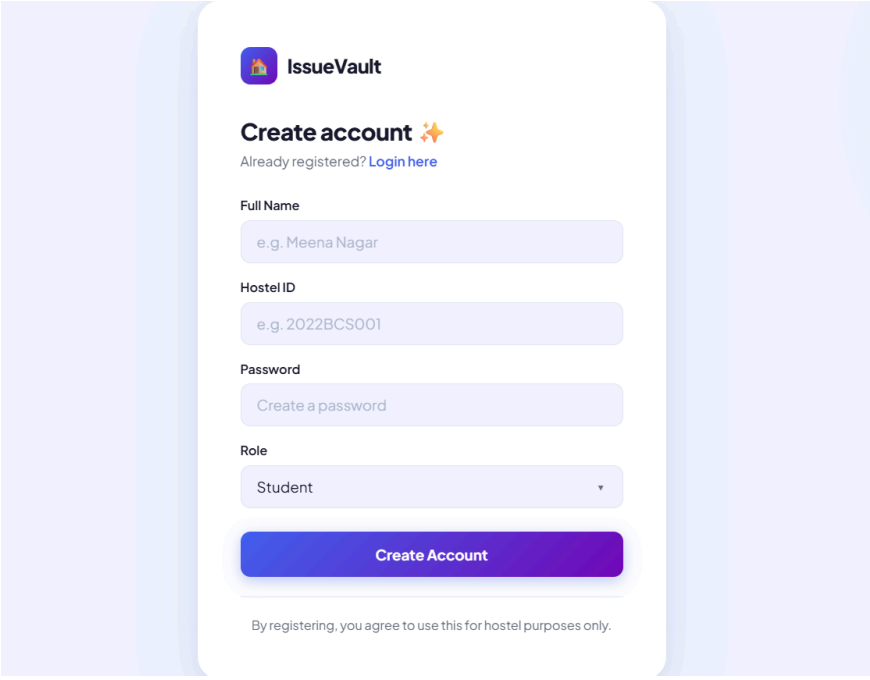


Figure 5.3: Complaint Submission Page

This page allows students to submit complaints by entering details such as complaint title and description. The system validates the input and stores the complaint in the database with a timestamp.

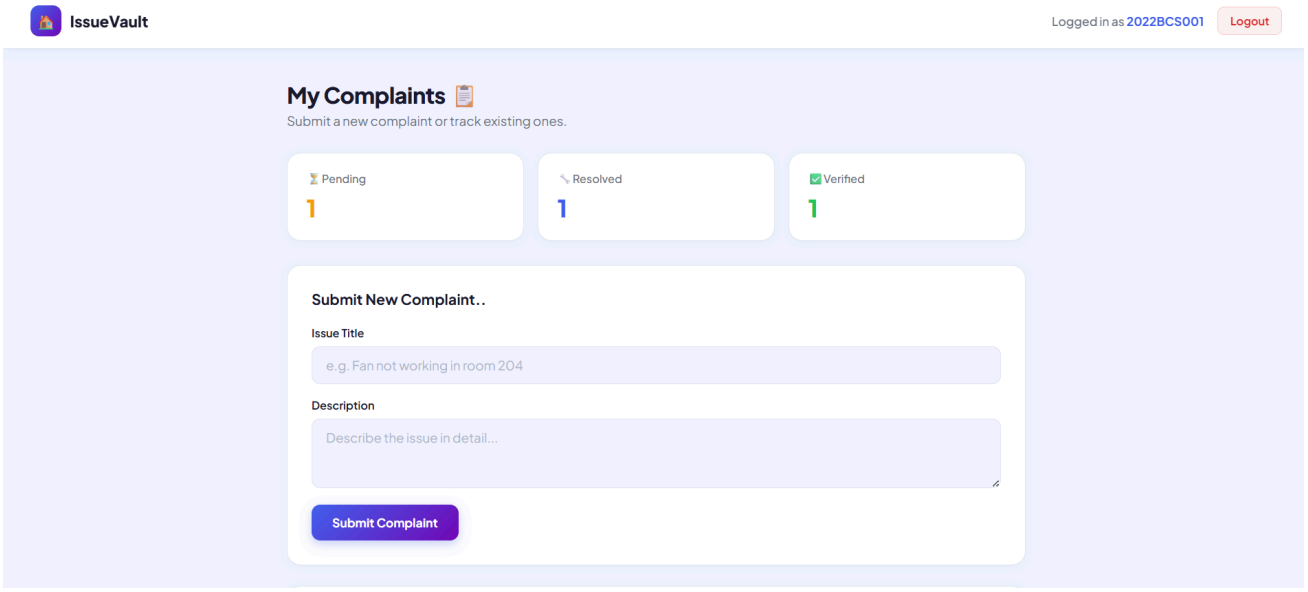


Figure 5.4: Admin Dashboard

The admin dashboard displays all submitted complaints in a structured format. The administrator can view complaint details and update their status after resolution.

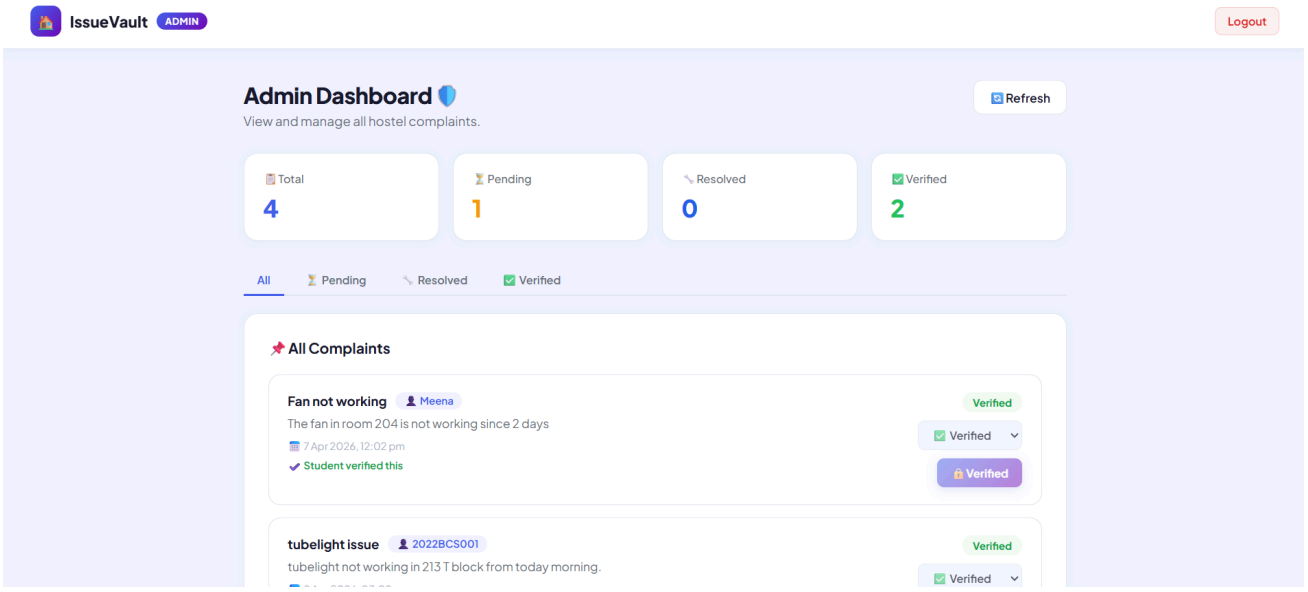
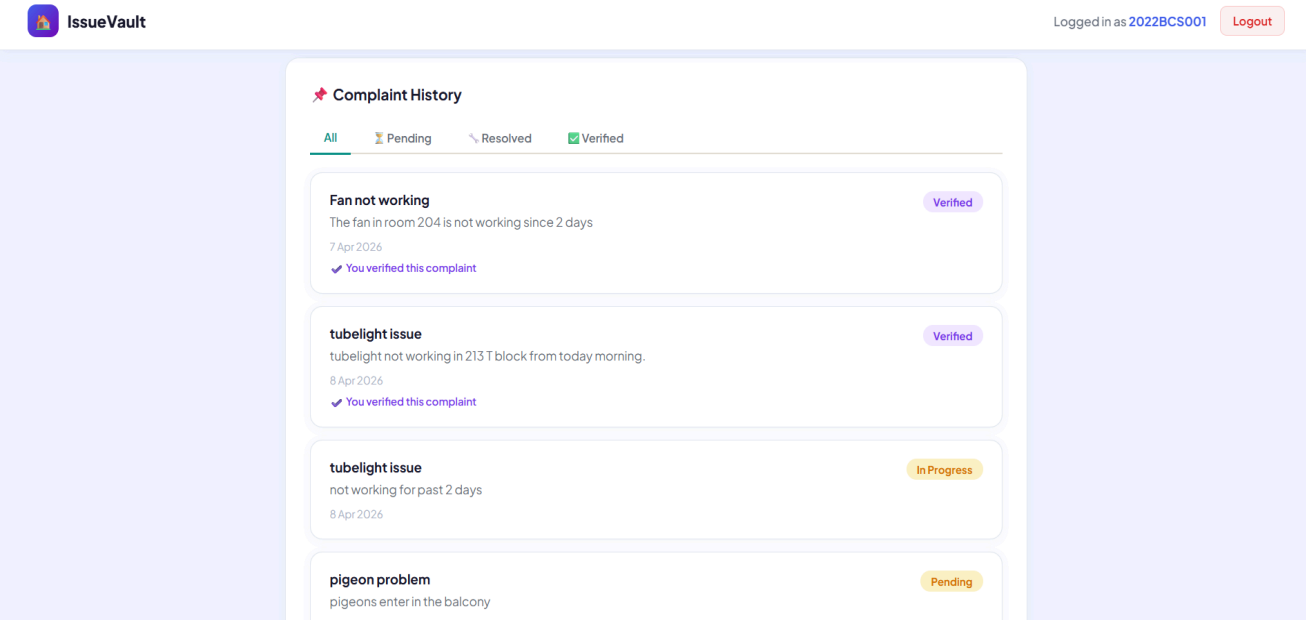


Figure 5.5: Complaint Status Page

This page allows students to track the status of their submitted complaints. It shows whether a complaint is Pending, Resolved, Verified, or Locked after final confirmation.



Chapter - 6

SUMMARY AND CONCLUSION

6.1 Summary

The IssueVault – Hostel Issue Reporting and Monitoring System is a web-based application designed to simplify the process of reporting, tracking, and resolving hostel-related issues. The system provides a structured platform where students can easily submit complaints, and administrators can efficiently monitor and update their status.

The system was developed using HTML, CSS, and JavaScript for the frontend, Node.js and Express.js for backend processing, and MongoDB for database management. It follows a modular design approach, ensuring separation of concerns and easier maintenance.

The application includes key features such as user authentication, complaint submission, complaint tracking, and status verification. A unique feature of the system is the verification mechanism, where students confirm resolution of complaints before final closure, after which the complaint is locked to maintain data integrity.

6.2 Conclusion

The IssueVault system successfully addresses the problems associated with traditional manual complaint handling systems in hostel environments. It improves communication between students and administrators, ensures transparency in complaint tracking, and reduces delays in issue resolution.

By digitizing the entire process, the system minimizes manual effort and provides a centralized platform for managing complaints efficiently. The inclusion of status tracking and verification enhances accountability and ensures that issues are properly resolved before closure.

Overall, the system is efficient, user-friendly, and scalable, making it suitable for deployment in hostel or institutional environments. It achieves its objective of providing a simple yet effective issue reporting and monitoring solution.

Chapter - 7

FUTURE SCOPE

The IssueVault system is designed as a scalable and flexible web application, allowing several enhancements in the future to improve its functionality and user experience.

1. Real-Time Notifications

The system can be enhanced with real-time notifications to inform users instantly about complaint status updates using email or in-app alerts.

2. Role-Based Access Control

Advanced role-based access can be implemented to define more user roles such as hostel wardens, maintenance staff, and supervisors with different permissions.

3. Image Upload Feature

Users can be allowed to upload images related to complaints, which will help administrators better understand and resolve issues quickly.

4. Mobile Application Development

A mobile version of the system can be developed to improve accessibility and allow users to report and track complaints more conveniently.

5. Analytics and Reporting

The system can include dashboards for administrators to analyze complaint trends, resolution time, and performance metrics.

Appendix A: Glossary

- **API:** Application Programming Interface
- **Database:** Structured data storage system
- **Frontend:** User interface of the application
- **Backend:** Server-side logic and processing
- **HTTP:** Communication protocol
- **JWT:** JSON Web Token used for secure authentication
- **User:** Any person who interacts with the system (student or admin)
- **Administrator:** A user with privileges to monitor and update issues
- **Issue:** A problem reported by a user for resolution
- **Status:** Current state of an issue (e.g., Pending, Resolved)
- **Authentication:** Process of verifying user identity

- **Authorization:** Process of granting access to specific functionalities.
- **Validation:** Checking input data for correctness before processing
- **Timestamp:** Date and time when an issue is created or updated

Appendix B: To Be Determined List

- Advanced authentication system (TBD)
- Real-time Notification system (TBD)
- Role-based access control (TBD)
- Image Uploading

BIBLIOGRAPHY

The following sources were referred to during the development of the IssueVault – Hostel Issue Reporting and Monitoring System:

Books

- Roger S. Pressman, *Software Engineering: A Practitioner's Approach*, McGraw-Hill Education.
- Ian Sommerville, *Software Engineering*, Pearson Education.

Web References

- https://www.w3schools.com (HTML, CSS, JavaScript tutorials)
- https://developer.mozilla.org (Web development documentation)
- https://nodejs.org (Node.js official documentation)
- https://expressjs.com (Express.js framework documentation)
- https://www.mongodb.com (MongoDB official documentation)

Tools and Technologies

- Visual Studio Code – Code editor used for development
- MongoDB Compass – Database visualization tool
- Google Chrome – Browser for testing the application
- Postman – API testing tool